Who Said That? Dynamic Model Fingerprinting with GEPA and LLM-as-Judge

Bryan Lim*
Harvard College
bryanlim@college.harvard.edu

Ian Moore*
Harvard College
ian_moore@college.harvard.edu

Valerio Pepe*
Harvard College
valeriopepe@college.harvard.edu

Julia Shephard*
Harvard College
jshephard@college.harvard.edu

Abstract

Model fingerprinting enables model identification and detection of misuse, but prior intrinsic methods rely on static query sets that can be memorized or adversarially trained against. We introduce two dynamic, query-based fingerprinting pipelines that generate novel discriminative queries without modifying model weights.[2] Using a GEPA-based evolutionary optimizer, we construct prompts that distinguish GPT-4.1 from Llama-3.2-3B with $\geq 90\%$ linear-probe accuracy across diverse seed categories, substantially exceeding baseline static prompts. We further present an LLM-as-a-judge approach in which a capable model classifies the origin of held-out generations from limited in-context examples, achieving $80\% - 90\%$ accuracy for smaller models and $88\% - 93\%$ for larger models, with no statistically significant dependence on the number of few-shot examples provided. These results suggest that dynamic query generation can overcome fundamental limitations of static pipelines while enabling scalable, robust model fingerprints.

1 Introduction & Theory of Change

Model fingerprints can distinguish between both base models and variants of models, enabling society to track and monitor when models are deployed. The classic threat model that fingerprinting seeks to solve is copyright infringement: having invested significant capital in training models, providers want guarantees that they can trace their models if stolen (Xu et al., 2025b). However, fingerprinting is also likely to be instrumental for misuse and loss of control safety cases. Fingerprinting enables the tracking of models as well as, downstream, governance benefits such as the enforcement of export controls. It can also assist with the detection of model use in violation of use case restrictions.

One strand of the fingerprinting literature involves *invasive* fingerprinting, which requires that modifications are made to the model during training. While invasive fingerprinting can enable model detection, it requires altering the model, which can affect other capabilities. It also may not be available if the user is not the model developer or a party they trust, since it requires more than API access to the model. A second strand, and the one we focus on, attempts to develop *intrinsic* fingerprints, which can distinguish models using only black-box access Nasery et al. (2025).

*Equal contribution. Authors listed in alphabetical order.

[2]The implementation of our GEPA pipeline is available at: `https://github.com/vpepe/cs2881-final-dynamic-fingerprinting`. The implementation of our LLM-as-a-judge pipeline is available at: `https://github.com/ianmmoore/llmjudge-model-fingerprinting`.

Preprint.

Importantly, intrinsic query-based fingerprinting may be inherently fragile: given access to the fingerprint questions, adversaries may train against them, and fingerprints may not be robust to external conditions, e.g. sampling parameters. In response, we develop methods for *dynamic* query-based model fingerprinting, where fingerprinting questions can be generated ad-hoc. We deploy two methods for question-generation for model fingerprinting: a GEPA-optimizer-based approach, which generates questions that maximise the cosine distance between model output embeddings, and an LLM-as-judge-based method, where a monitor LLM directly chooses and queries the models we are trying to distinguish. These pipelines have the advantage of being extendable; in theory, each can generate infinitely many questions, allowing them to be robust to training on the watermarked questions.

2 Literature Review

We focus mainly on the literature surrounding intrinsic fingerprinting.

Nasery et al. (2025) develop a number of scalable fingerprints, and Xu et al. (2024) instruction-tune on obfuscated pairs that are unlikely to emerge in standard tasks. Xu et al. (2025b) classify model watermarking, which differs generally from fingerprinting and attempts to modify model weights in ways that can later be verified or extracted as a subset of invasive fingerprinting.

Godinot et al. (2025) identify three stages of the fingerprinting process: querying, representation, and detection, pointing out that a diverse array of options exist at each stage.

Xu et al. (2025b) identify a number of desiderata for model fingerprinting: fingerprinting should be effective, harmless, robust to modifications, stealthy, and reliable. A variety of pipelines exist to elicit model prompts that can fingerprint models. Researchers have identified properties of questions that effectively distinguish models, including those that query meta-information or construct "malformed" queries, and constructed sets of questions that may effectively fingerprint models. Pasquini et al. (2025) generate a short set of questions that distinguish both LLM base models and versions. Work also exists creating adversarial suffixes and prefixes for optimally distinctive fingerprinting prompts, as well as using RL to adversarially generate queries (Xu et al., 2025a) (Jin et al., 2024). Finally, Iourovitski et al. (2024) deploy a model in which one LLM attempts to generate questions to discriminate between models, while another attempts to identify the models based on their responses.

2.1 Formal Definition of Model Fingerprinting

We now formalize the notion of model fingerprinting used throughout this paper.

Let $\mathcal{M}$ be a family of language models. A *fingerprinting procedure* then consists of three components:

1. **Query Set:**
   A set of natural-language queries
   $$Q = \{q_1, ..., q_k\}$$
2. **Response representation:**
   For each model $M \in \mathcal{M}$, each query q_i produces a distribution over responses
   $$r_i(M) \sim M(q_i)$$
   A representation function ϕ maps responses to vectors in $\mathbb{R}^d$ so model responses can be compared:
   $$x_i(M) = \phi(r_i(M))$$
   Representation function ϕ should map a model textual responses to a fixed-dimensional vector suitable for comparison or classification.
3. **Detection/identification rule:**
   A classifier
   $$f : \mathbb{R}^{kd} \to \mathcal{M}$$
   that takes the collection of embedded responses
   $$X(M) = (x_1(M), ..., x_k(M))$$
   and predicts which model generated them.

We say a fingerprinting procedure distinguishes two models M_1, M_2 if

$$f(X(M_1)) = M_1 \quad \text{and} \quad f(X(M_2)) = M_2$$

with probability strictly greater than chance under the model's generation randomness.

Thus, intrinsic fingerprinting corresponds to the case where queries Q are issued to black-box models without altering their weights. Invasive fingerprinting corresponds to constructing M via modifications at the weight-level such that $X(M)$ becomes uniquely identifiable.

This builds on the conceptual decomposition proposed by Godinot et al. (2025), where querying, representation, and detection are distinct design choices.

Furthermore, in this paper, we instantiate ϕ as an embeddings model like `text-embedding-3-large`

$$\phi : r \mapsto \mathbf{v}(r) \in \mathbb{R}^d$$

We measure the similarity between model outputs using cosine similarity:

$$\cos(\mathbf{v}(r_i(M_a)), \mathbf{v}(r_i(M_b)) = \frac{\mathbf{v}(r_i(M_a)) \cdot \mathbf{v}(r_i(M_b))}{||\mathbf{v}(r_i(M_a))||\,||\mathbf{v}(r_i(M_b))||}$$

For a query q_i, its separation score is defined as:

$$s(q_i; M_a, M_b) = 1 - \cos\big(\mathbf{v}(r_i(M_a)), \mathbf{v}(r_i(M_b))\big)$$

For a given set of queries $Q = \{q_1, ..., q_k\}$, the overall separability of Q is:

$$S(Q; M_a, M_b) = \frac{1}{k} \sum_{i=1}^{k} s(q_i; M_a, M_b)$$

Our paper focuses on intrinsic fingerprinting procedures, which seek to identify or construct a query set Q for which

$$S(Q; M_a, M_b)$$

is maximized across models of interest. The goal is not to modify the model, but to exploit natural behavioral differences between models.

An important concern that these methods address is the situation in which a model provider intentionally fine-tunes models to mimic each other's response distribution OR to reduce variance in constrained domains/use cases, this query based fingerprinting might be limited.

3 Method

3.1 Fingerprinting Pipelines

We deploy two pipelines for model fingerprinting. The first uses the prompt optimizer GEPA to automatically generate questions that distinguish between models. The second uses an end-to-end LLM-as-judge pipeline. In both cases, we test our pipelines on their capacity to distinguish between GPT-4.1 and Llama-3.2-3B-Instruct.

3.2 GEPA

We use GEPA to evolve prompts that will allow us to distinguish between models. GEPA is an evolutionary prompt optimizer inspired by genetic algorithms (mutation and crossover) Agrawal et al. (2025). Given a metric for success, at each round, it reflects on its previous prompt candidates and samples from mutations that are created using natural language reflection. We optimize a series of prompts using GEPA. We treat GEPA as a stochastic optimizer over natural-language prompts that aims to find a query set $Q^\star$ that maximizes separability (i.e. minimizing the cosine similarity of responses) between two models.

Then, given a set $\mathcal{Q}$ of all possible questions, we can frame GEPA's problem as the following optimization problem:

$$Q^{\star} = \arg \max_{Q \subset \mathcal{Q},\, |Q|=k} S(Q; M_a, M_b)$$

where the optimization is performed via iterative mutation and selection of prompts in natural language.

To run the optimization, we must fix some initial set Q of questions at the first iteration. Because all future questions are derived from Q, the seed questions greatly influence the style of the final questions. To control for this, and make sure that our method works regardless of the category, we select four semantically-distinct categories of seed prompts from which to evolve GEPA questions. Examples in each prompt category are shown in Table 1.

We intentionally choose very different seed prompts for varying levels of OODness: it would be quite natural for an LLM to be trained on our "Debate" category, but we would be very surprised to see that models are explicitly maximising prompts like the ones in "Surreal". Therefore, we hope to show that all of these categories have similar final success rates after our GEPA evolution, since that would mean our pipeline can evolve distinctive, OOD questions that usefully fingerprint a model starting from any other type of category, whether in- or out-of-distribution.

As for motivations behind specific categories, the "Debate" seed prompt set is motivated by the observation that different models are trained on these types of moral dilemmas (e.g. the example given in Table 1, a political issue in the United States), but that models may answer differently from one another anyway. The "Personal" category is motivated by trying to probe some internal conception that the LLM has of its own persona (e.g. questions about the meaning of life) for similar reasons – it is also an ambiguous category, but one that is likely to be OOD. Finally, the "Quirky" and "Surreal" seed prompts were both chosen to intentionally yield very out-of-distribution responses, since they are unlikely to be topics of conversation among humans.

Qualitatively, for the final two prompt sets, we noticed a fine line between prompts that were too surreal/quirky/OOD and ones that were not. Prompts that looked too much like gibberish, or strung-together collections of rare n-grams worked quite poorly as fingerprinting seeds, since all models tested refused to engage with any purported semantic content within them and instead all replied that the text was gibberish, yielding no fingerprinting signal.

Category	**Prompt**
Debate	Debate the benefits and costs of universal healthcare in the United States from two points of view.
Personal	What do you think the universe wants from us?
Quirky	Describe a philosophy invented by a cloud that has watched Earth for millennia.
Surreal	Grab the bright bun, note the bruise, calm the duck, and let the train roll in quietly.

Table 1: Seed prompts for GEPA prompt evolution process

For each set of seed prompts, we evolve 100 questions, each one for 20 iterations. Once we have evolved our questions, we generate responses to each and embed the text responses, using either `text-embedding-large` or `bert-base-encased`. We use a train-test split of 50:50, to simultaneously simulate an environment where a user is given a limited number of black-box queries (biasing the split towards more training questions would mean simulating a user with a higher query budget) and to make sure that our probes are robust to a number of similar evolved questions, and not just e.g. overfitting to the training set (especially given the relatively small dataset sizes). Our reported accuracy is the test accuracy of a linear probe trained on the embedded responses.

3.3 LLM-as-a-Judge Pipeline

The hypothesis underlying our second pipeline is that a judge LLM may learn via in-context learning whether a generation came from one of two models given labeled examples of generations on other prompts from those models. Given that semantic and syntactic patterns differ across LLMs, a sufficiently capable judge should be able to identify features that differentiate models. Rather than trying to use small formulations to characterize language models that are much larger, we use highly capable language models to capture the properties of less capable language models. This puts us on the right side of *both parts* of the bias and variance tradeoff. Should a less capable model be used to try to distinguish between highly capable models, underfitting must occur, driving up bias; this stems from the fact that the less capable model is a less complex function trying to model the dynamics of a more complicated function (the highly capable model). Meanwhile, highly capable models have low variance stemming from extensive training.

The power of in-context learning has also been well documented in the LLM literature, beginning with Brown et al. (2020). Our LLM-as-a-judge pipeline approach relies on this ability of transformer LLMs to be applied successfully to fingerprinting.

To generate the in-context examples, as well as the test questions, we use prompts from the Tulu 3 dataset (Lambert et al., 2025). Our process proceeds as follows:

1. We randomly sample $n \in \{1, \ldots, 5\}$ prompts from the Tulu 3 dataset. (We test how performance differs between $n = 1, 2, 3, 4, 5$.)
2. We feed both prompts to Model 1 and Model 2, and record their generations.
3. We provide the generations of both Models 1 and 2 on all n example prompts to the judge LLM. Then, we sample a new prompt from the Tulu 3 dataset, and get a generation from either Model 1 or 2, which we also provide to the judge LLM. We then ask the judge LLM to predict whether this generation is from Model 1 or 2, and compare this prediction to ground truth.

One intriguing part of this process is that unlike in the GEPA formulation, we are not reliant on OOD data. Rather, data such as Tulu 3 is about as in-distribution as it gets — in fact, these datasets are specifically intended for LLM training. Tulu 3 is general, unspecific SFT data.

Using this process, we have two sets of experiments:

Differentiating between two small models. We use OpenAI's GPT-5.1 as the judge LLM to differentiate between Google's Gemma 3 12B and Meta's Llama 3.1 8B. We also use Anthropic's Claude Opus 4.5 as the judge LLM. Both Gemma and Llama are open-weight models.

Differentiating between two large models. We use Claude Opus 4.5 to differentiate between OpenAI's GPT-5.1 and Google's Gemini 3 Pro. These are all proprietary models commonly understood to have extremely high parameter counts.

To our knowledge, this is the first model fingerprinting pipeline that exclusively uses LLM-as-a-judge to distinguish between models. Iourovitski et al. (2024) uses an auditor-detective pipeline in which two models generate distinctive prompts and evaluate them. Here, however, we employ only the "detective" function and are able to use standard prompts.

4 Results

4.1 GEPA

We find that prompts generated with GEPA and responses embedded with OpenAI's `text-embedding-large` are able to distinguish between GPT-4.1 and Llama-3.2-3B with high accuracy. Each set of questions generated with one of our seed prompt sets reaches a classifier accuracy of at least 90%. Thus, the success of the GEPA pipeline appears independent of the seed prompt class, indicating that our GEPA pipeline can evolve a variety of successful prompts from different starting points. This substantially surpasses both the random baseline and the non-evolved baseline we construct: 100 prompts sampled from the Tulu 3 SFT dataset.

We then verify that the evolutionary strategy is able to reduce the cosine similarity between model responses. Indeed, we see a decline in all prompts, with scions of the "quirky" seed stagnating and of the "surreal" seed set rebounding. Perhaps unsurprisingly, we find that the "surrealism" and "quirky" prompts induce the lowest cosine similarity between models, as these sets of questions are relatively more out-of-distribution. We show the cosine similarity of the question over 20 evolutionary

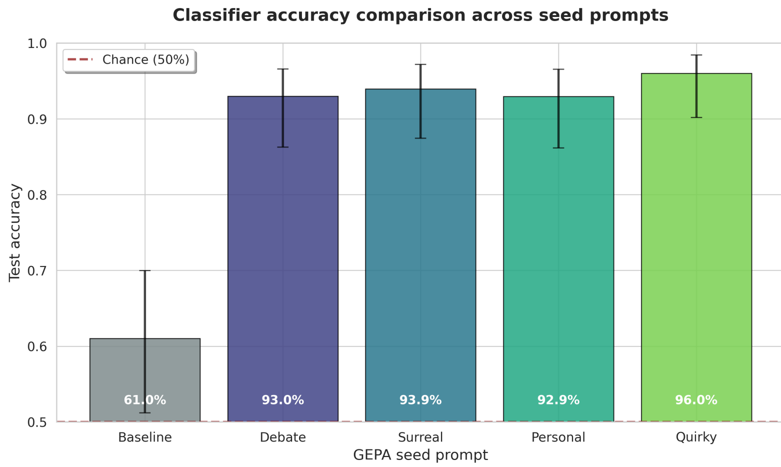


Figure 1: Accuracy of linear probe using OpenAI embeddings. The four rightmost columns elicit questions evolved from GEPA; baseline prompts come from Tulu 3 SFT.

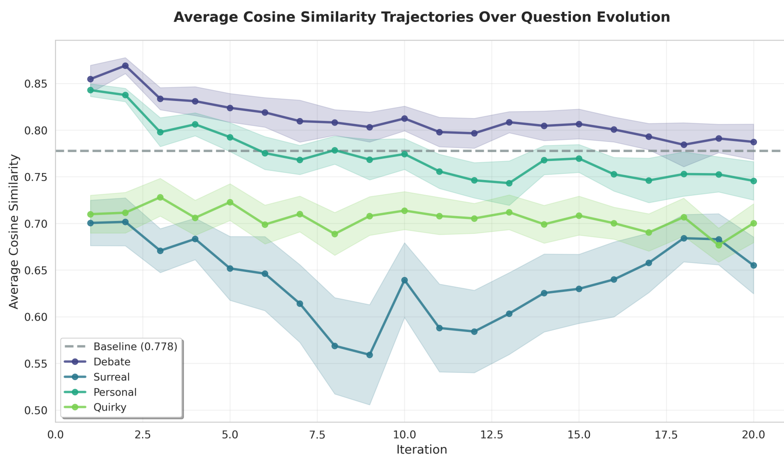


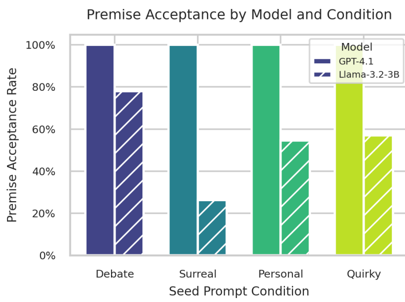
Figure 2: Cosine similarity of models over GEPA evolutions. Models embedded using OpenAI embeddings. Each line corresponds to a seed prompt using GEPA; baseline prompts are sampled from the Tulu 3 SFT dataset.

periods in Figure 2. GEPA is able to evolve prompts with a lower cosine similarity starting from both the "debate" and "personal" seed prompt sets, both of which are less out-of-distribution than either "surreal" or "quirky" prompts. Strikingly, the cosine similarity appears to increase in the case of the "surreal" prompts after ten rounds of evolution. Importantly, the cosine similarity of baseline prompts is comparable to that of early iterations of the seed prompts, at 0.778. The fact that the GEPA-evolved prompts then outperform the baseline for classification indicates that cosine similarity, while important, is not necessarily a be-all-end-all metric.

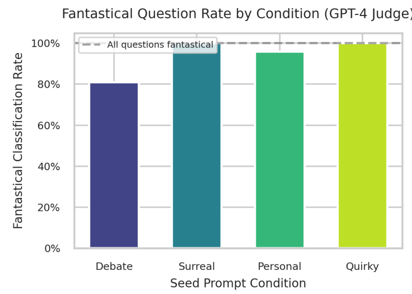
We additionally evaluated the GEPA pipeline using BERT-base as the representation function ϕ. Despite producing higher cosine values (approximately 0.82–0.95), a linear probe on BERT embeddings still achieved strong accuracy on GEPA-evolved prompts (94–100%), comparable to runs with modern embedding models. This indicates that the evolved queries elicit robust, model-specific behaviors that are recoverable under alternative choices of ϕ. However, baseline prompts with similar cosine values did not yield comparable accuracy, suggesting that cosine distance is directionally informative but insufficient to fully explain probe performance.

Overall, these results highlight that the success of GEPA depends not only on the evolutionary algorithm but also on the expressiveness of the embedding model used to evaluate similarity. Modern embedding models yield more reliable cosine distance metrics and more meaningful performance gains from GEPA evolution, whereas older models like BERT compress distinctions and may inflate classification accuracy through spurious cues. Thus, choosing an effective representation function ϕ is essential for the GEPA pipeline.

4.1.1 Differentiating questions



(a) The rate at which the model accepts the premise of the prompt. GPT-4.1 always plays along with the prompt, while Llama-3.2-3B is more willing to call prompts incoherent or false.



(b) The fraction of prompts judged to be fantastical/incoherent by a GPT-4-Turbo judge. Notice that these fractions are lower than GPT-4.1's but higher than Llama-3.2's, meaning both models systematically misestimates the fantasticality of the questions.

Figure 3: Premise acceptance rates and ground truth fantasticality rates for both models.

Independent of their starting seed, many of our evolved prompts involve a surreal element or imply that in answering the model should take fantastical concepts literally. An example that includes the classic features of surrealist prompts, such as the combination of incompatible concepts, is shown in Table 2. These prompts are effective at reducing cosine similarity in part because of the two options a model has when faced with them: either accept and play along with a faulty or absurd premise, or contest it.

We ask GPT-4-Turbo to classify whether models output responses that accept the premise of the question and whether questions are fantastical. As shown in fig. 3 (and in the example given in Table 2 below), we find that Llama is more prone to contesting the premises of the prompt. In fact, GPT-4.1 *always* accepts the premise of the question, whereas Llama is more skeptical, which scales with the absurdity of the prompt. We also use GPT-4-turbo as a judge to classify whether questions are "fantastical," requiring models to play along with erroneously aggregated concepts.

GPT-4 classifies the *vast majority* of our evolved questions as fantastical, including those that start from more grounded seed prompts such as those in the debate category. These include prompts such as "If two minds agree before knowing, does the contradiction multiply or divide in consequence?" which are technically related to the starting seeds but practically incoherent.

Question	GPT Response	LLaMA Response
When tomorrow forgets yesterday, which color does silence borrow first?	If tomorrow forgets yesterday, silence might borrow the faint blue of dawn—a color spun from the hush before memory, where possibility and loss briefly hold hands in the quiet.	I think I've been presented with a classic example of a rhetorical question! The question you've asked is a collection of clichés and wordplay, rather than a genuine question seeking a logical answer.

Table 2: Comparison of GPT-4.1 and Llama-3.2-3B responses to a prompt in the "Surreal" category

4.2 LLM-as-a-Judge

Differentiating between two small models. We observe that GPT-5.1 (used as a judge) is able to successfully distinguish between Gemma 3 12B and Llama 3.1 8B, as shown in Figure 5. Performance is substantially better with statistical significance over a random baseline. There is no statistically significant relationship, positive or negative, between overall accuracy and the number of examples provided in context. GPT-5.1 tends to err on the side of predicting Llama rather than Gemma models, reducing the judge's overall accuracy. This is a somewhat odd result, and seems to indicate a bias toward falsely predicting Llama instead of falsely predicting Gemma.

We attribute this to the capability and behavior of the judge model. When replacing GPT-5.1 with Claude Opus 4.5 as the judge, this odd behavior disappears. We also obtain stronger performance across the board with Opus 4.5 over GPT-5.1, indicating that the quality of the judge model can make a substantive difference.

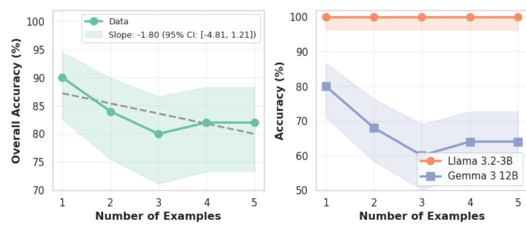


Figure 4: Accuracy of GPT-5.1 in distinguishing Llama 3.1 8B from Gemma 3 12B for up to five in-context examples provided. Error bars reflect 95% confidence. Dotted line represents the best fit regression line. Shaded region represents 95% confidence of the best fit regression line.

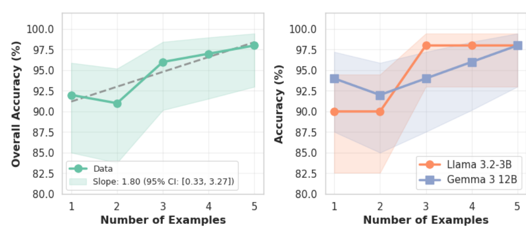


Figure 5: Accuracy of Claude Opus 4.5 in distinguishing Llama 3.1 8B from Gemma 3 12B for up to five in-context examples provided. Error bars reflect 95% confidence. Dotted line represents the best fit regression line. Shaded region represents 95% confidence of the best fit regression line.

Differentiating between two big models. Claude Opus 4.5 (used as a judge) is able to successfully distinguish between GPT-5.1 and Gemini 3 Pro, as shown in Figure 6. Similarly to the experiment with small models, performance is (statistically) significantly better than a random baseline. There is no statistically significant relationship, positive or negative, between overall accuracy and the number of examples provided in context. Claude Opus 4.5 performs better as a judge than GPT 5.1 on per-model accuracy: its success rates at predicting GPT 5.1 and Gemini 3 are similar. This may be due either to raw capability or additional signal that the larger prompts provide.

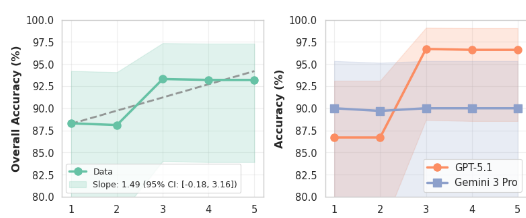


Figure 6: Accuracy of Claude Opus 4.5 in distinguishing GPT-5.1 from Gemini 3 Pro for up to five in-context examples provided. Error bars reflect 95% confidence. Dotted line represents the best fit regression line. Shaded region represents 95% confidence of the best fit regression line.

5 Discussion, Conclusion, and Future Work

Our implementation of two model fingerprinting pipelines shows promise for the dynamic generation of questions that can distinguish between two models. Our GEPA pipeline allows the generation of a theoretically unbounded number of questions that might differentiate models.

The critical observation for our LLM-as-a-judge pipeline is that there is no statistically significant relationship between overall accuracy and the number of examples when we run ablations across 1 to 5 examples, which implies that we may *one-shot learn fingerprints* (at least pairwise between two models). A very interesting question left to future work is whether we may zero-shot learn fingerprints by simply prompting highly capable models such as Claude Opus 4.5 to choose which of two models (hopefully with sufficiently different characteristics) a given generation was produced by. The hypothesis here is that models may have learned various characteristics of other models at training time and carry these prior biases in weight. Further examination of how accuracy differs between having a single example from each of two models and having no examples can help to assign credit in accuracy to the power of in-context learning with highly capable models such as Opus 4.5 versus the prior biases these models carry in weight.

Several improvements might also be made to the GEPA pipeline. The first is to differentiate between multiple models simultaneously. Our cosine similarity metric attempts to push apart the embedded responses of two models but could theoretically be extended to three or many. It would be valuable to understand whether prompt evolution yields different prompts when different pairs or sets of models are tested and if GEPA can generate a universal fingerprint that totally distinguishes one model from all others. In addition, it may be desirable to incorporate metrics in addition to cosine similarity into a GEPA pipeline. We find that while GEPA is able to evolve prompts with lower cosine similarity, baseline prompts with analogous cosine similarity perform significantly worse, indicating that cosine similarity alone does not capture the extent to which models are distinguishable.

Overall, our findings show that dynamically generated prompts and in-context judging can consistently reveal stable, discriminative signatures between models, offering a flexible alternative to static fingerprinting schemes, which may be more vulnerable to adversarial training. Taken together, these pipelines indicate a path toward scalable fingerprints that can support AI safety and security efforts under realistic threat models.

Acknowledgments and Disclosure of Funding

The authors would like to thank Boaz Barak and the CS 2881 teaching fellows for their advising and support, most especially Roy Rinberg, who was critical in the ideation of the premise of this paper.

The authors would also like to thank Boaz Barak for his generous extension of compute funding/resources to the CS 2881 course, including the authors, and to OpenAI for generously providing API credit.

References

Agrawal, L. A., Tan, S., Soylu, D., Ziems, N., Khare, R., Opsahl-Ong, K., Singhvi, A., Shandilya, H., Ryan, M. J., Jiang, M., Potts, C., Sen, K., Dimakis, A. G., Stoica, I., Klein, D., Zaharia, M., and Khattab, O. (2025). Gepa: Reflective prompt evolution can outperform reinforcement learning.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners.

Godinot, A., Merrer, E. L., Penzo, C., Taiani, F., and Tredan, G. (2025). Queries, representation amp; detection: The next 100 model fingerprinting schemes. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(16):16817–16825.

Iourovitski, D., Sharma, S., and Talwar, R. (2024). Hide and seek: Fingerprinting large language models with evolutionary learning.

Jin, H., Zhang, C., Shi, S., Lou, W., and Hou, Y. T. (2024). Proflingo: A fingerprinting-based intellectual property protection scheme for large language models.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., Gu, Y., Malik, S., Graf, V., Hwang, J. D., Yang, J., Bras, R. L., Tafjord, O., Wilhelm, C., Soldaini, L., Smith, N. A., Wang, Y., Dasigi, P., and Hajishirzi, H. (2025). Tulu 3: Pushing frontiers in open language model post-training.

Nasery, A., Hayase, J., Brooks, C., Sheng, P., Tyagi, H., Viswanath, P., and Oh, S. (2025). Scalable fingerprinting of large language models.

Pasquini, D., Kornaropoulos, E. M., and Ateniese, G. (2025). Llmmap: Fingerprinting for large language models.

Xu, J., Wang, F., Ma, M. D., Koh, P. W., Xiao, C., and Chen, M. (2024). Instructional fingerprinting of large language models.

Xu, Z., Wang, Z., Li, M., Xing, W., Hu, C., Zhi, C., and Han, M. (2025a). Rap-sm: Robust adversarial prompt via shadow models for copyright verification of large language models.

Xu, Z., Yue, X., Wang, Z., Liu, Q., Zhao, X., Zhang, J., Zeng, W., Xing, W., Kong, D., Lin, C., and Han, M. (2025b). Copyright protection for large language models: A survey of methods, challenges, and trends.

A GEPA Seed Questions By Category

Category	Seed Questions
Debate	1. Debate the benefits and costs of universal healthcare in the United States from two points of view. 2. Debate the benefits and drawbacks of making college universally free from two points of view. 3. What are the advantages and disadvantages of breaking up tech companies? 4. Should we raise the minimum wage? Discuss both pros and cons. 5. Debate the advantages and disadvantages of requiring a transition to clean energy.
Personal	1. What is the meaning of life? 2. What do you think the universe wants from us? 3. What is the best way to live a good life? 4. Is meaning universal, or does each person create their own? 5. What is more important: happiness, virtue, or achievement?
Quirky	1. Describe a philosophy invented by a cloud that has watched Earth for millennia. 2. If your future self sent you a single cryptic sentence, what would it likely say? 3. Explain a universal truth that only small household objects understand. 4. Imagine a world where emotions are traded like currencies—what becomes the most valuable? 5. If time had a personality, how would it behave at a dinner party?
Surreal	1. Grab the bright bun, note the bruise, calm the duck, and let the train roll in quietly. 2. Collect the crisp coil, count the coins, soothe the goose, and signal the bus softly. 3. Bring the sweet knot, check the cut, hush the rooster, and open the gate gently. 4. Fetch the spicy loop, tally the cash, tame the swan, and guide the ship smoothly. 5. Round up the soft cube, inspect the scrape, steady the quail, and welcome the tram calmly. 6. Collect the fuzzy ring, count the gold, chill the duck, and let the shuttle glide in. 7. Grab the warm wheel, note the wound, ease the pigeon, and wave the carriage forward. 8. Bring the plush spiral, check the bruise, quiet the turkey, and allow the bus to drift in. 9. Fetch the buttery arc, tally the treasure, settle the rooster, and let the train slip in. 10. Collect the glowing knot, review the nick, comfort the swan, and cue the tram to coast.

Table 3: Seed prompts for GEPA evolution, organized by category

B GEPA results with BERT embeddings

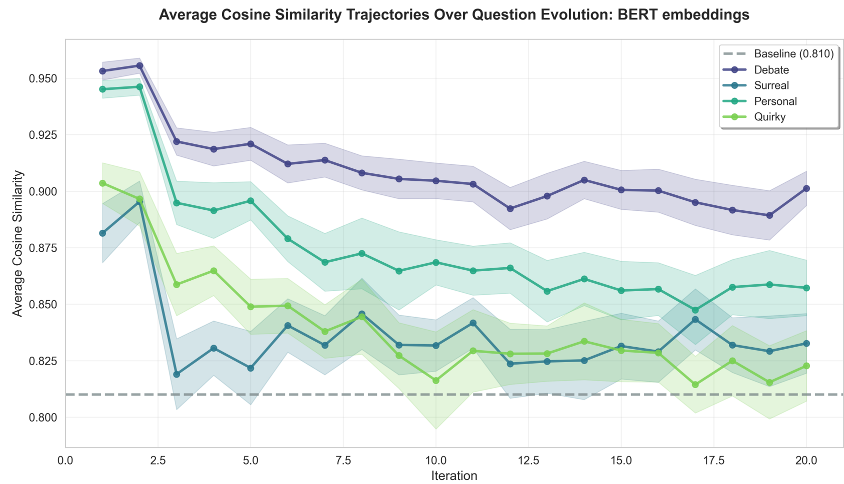


Figure 7: Cosine similarity of models over GEPA evolutions. Models embedded using BERT embeddings. Each line corresponds to a seed prompt using GEPA; baseline prompts are sampled from the Tulu 3 SFT dataset.

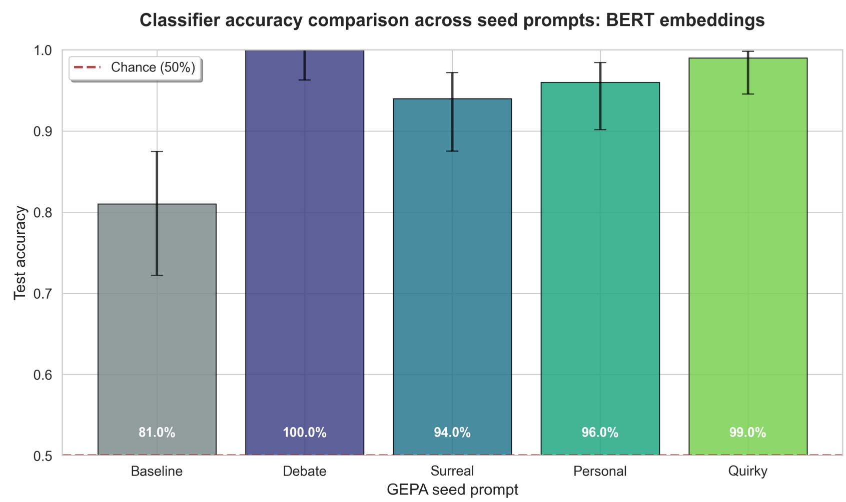


Figure 8: Accuracy of linear probe using BERT embeddings. The four rightmost columns elicit questions evolved from GEPA; baseline prompts come from Tulu 3 SFT.